Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 9

ConvNet Architecture Patterns

Four best practices that turn a working image model into a good one — and a mini Xception that beats the previous chapter's model with less than half the parameters.

Duration 2.5 hours

Instructor Rahman Indra Kesuma, S.Kom., M.Cs. · Prof. Bambang Riyanto Trilaksono

Source Chollet & Watson, Deep Learning with Python 3e — chapter 9

LEARNING OUTCOMES

Session Objectives

By the end of this session, participants can:

1. Explain architecture as a **choice of hypothesis space**, and why a good one reduces the search gradient descent has to do.
2. Apply the **modularity–hierarchy–reuse** formula to read any published ConvNet.
3. Explain the **vanishing gradient** problem and fix it with **residual connections**, including the 1×1 projection when shapes differ.
4. Say what **batch normalisation** does, where to put it, and why `use_bias=False` accompanies it.
5. Explain the assumption behind **depthwise separable convolutions**, and why they are not faster on a GPU despite using far fewer parameters.
6. Assemble all four into a **mini Xception**, and read the resulting parameter-count and accuracy change.
7. Say when a **Vision Transformer** is the better choice, and when it is not.

BOOK
Chapter 9 — full text

NOTEBOOK
01_residual_connections.ipynb

NOTEBOOK
02_batchnorm_and_separable.ipynb

NOTEBOOK
03_mini_xception.ipynb

NOTEBOOK
All chapter 9 notebooks

REPO
Author's official notebook

Architecture is the choice of hypothesis space

Like feature engineering, a good hypothesis space **encodes prior knowledge**. Using convolution layers means you already know that the relevant patterns are **translation invariant**

Why it is often the difference between success and failure

Bad choices cannot be trained away

The model gets stuck at suboptimal metrics, and **no amount of training data will save it**.

Good choices multiply your data

Learning accelerates and the model uses the available data efficiently, **reducing the need for a large dataset**.

> *A good model architecture is one that reduces the size of the search space, or otherwise makes it easier to converge to a good point of the search space.*
>
> Chollet & Watson, chapter 9 introduction

An honest note about how this knowledge is held

> The keyword is *intuitively*: **no one can give you a clear explanation of what works and what does not**. Experts rely on pattern matching acquired through practice.

But it is not *only* intuition. As in any engineering discipline there are **best practices**, and this chapter is four of them. Remember, too, that gradient descent is **a pretty stupid search process** — it needs all the help it can get.

01

Modularity, hierarchy, and reuse

The universal recipe for making a complex system simpler.

SECTION 9.1

The MHR formula

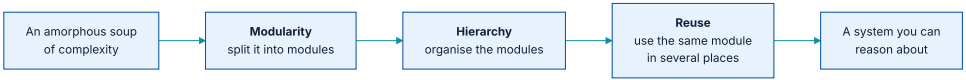


Figure 9.1 — the same recipe underlies a cathedral, your body, a navy, and the Keras codebase.

Software engineers already know this: an effective codebase is modular, hierarchical, and does not reimplement the same thing twice. Following those principles **is** software architecture.

SECTION 9.1

Deep learning is that recipe applied to optimisation

- Take a classic optimisation technique — **gradient descent over a continuous function space**.
- Structure the search space into **modules**: layers.
- Organise them into a **deep hierarchy** — often just a stack, the simplest kind.
- **Reuse** whatever you can: convolutions are entirely about reusing the same information at **different spatial locations**.

SECTION 9.1

Two patterns visible in every published ConvNet

Repeated blocks

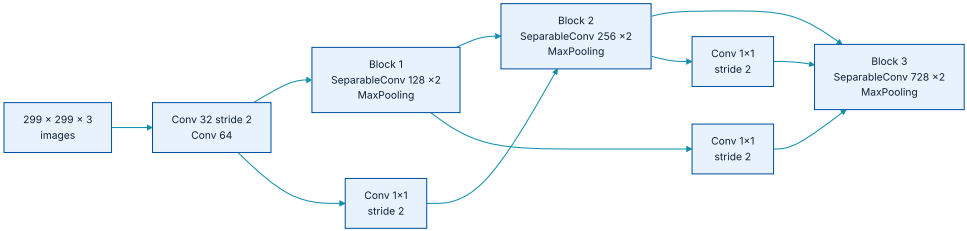
Popular architectures are structured not merely into layers but into **repeated groups of layers** — *blocks* or *modules*. Xception repeats a SeparableConv – SeparableConv – MaxPooling block.

A feature pyramid

Filter count **grows with depth** while feature-map size **shrinks**. You saw 32 → 64 → 128 in chapter 8; the same pattern runs through Xception.

Once you can see those two patterns, most architecture diagrams in papers become readable at a glance.

Xception, read with those two patterns



Repeated blocks, growing depth, and a 1x1 projection carrying each residual across the downsampling.

The spatial sizes fall 299 → 149 → 74 → 37 while the depth climbs 3 → 32 → 128 → 256 → 728. Everything else on this diagram is **the same block, repeated**

02

Residual connections

The game of telephone, and how to stop playing it.

SECTION 9.2

Backpropagation as a game of telephone

Each successive function introduces noise into the error signal travelling backwards.

SECTION 9.2

The vanishing gradient problem

> If the chain is too deep, that noise **overwhelms the gradient information** and backpropagation stops working. **The model will not train at all.**

The fix is to force each function in the chain to be **non-destructive** — to retain a noiseless version of what came in.

SECTION 9.2

A residual connection is an information shortcut

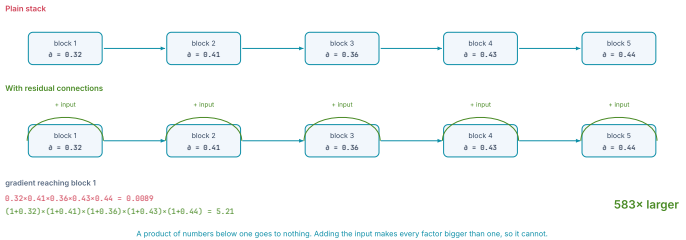


Figure 9.3 — the same five blocks, with and without the shortcut, and the two gradients they produce.

Same blocks, same derivatives — and the gradient reaching block 1 is hundreds of times larger. Introduced in 2015 with **ResNet** (He et al., Microsoft).

Three lines, and one condition

listing 9.1 — a residual connection PYTHON

```
x = ...                          # some input tensor
residual = x                     # save a reference: this is the residual
x = block(x)                     # this block may be destructive or noisy - fine
x = add([x, residual])           # the output now always retains the full input
```

The condition: adding requires **the block output and the residual to have the same shape**. That fails as soon as the block changes the filter count or pools — which is **most real blocks**

Case 1 — the block changes the filter count

listing 9.2 — projecting the residual PYTHON

```python
inputs = keras.Input(shape=(32, 32, 3))
x = layers.Conv2D(32, 3, activation="relu")(inputs)
residual = x                                          # 32 filters

x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)   # now 64 filters
residual = layers.Conv2D(64, 1)(residual)             # 1x1 projection to match

x = layers.add([x, residual])
```

`padding="same"` in the block avoids the spatial shrinkage a convolution would otherwise cause, **keeping the two shapes aligned**

Case 2 — the block pools

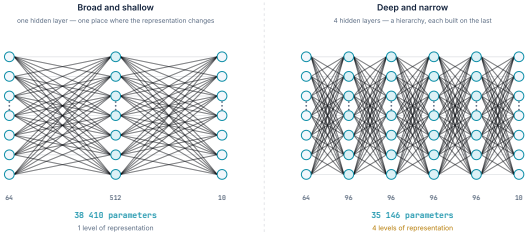
listing 9.3 — matching a max pooling layer PYTHON

```python
inputs = keras.Input(shape=(32, 32, 3))
x = layers.Conv2D(32, 3, activation="relu")(inputs)
residual = x

x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
x = layers.MaxPooling2D(2, padding="same")(x)           # halves the spatial size

residual = layers.Conv2D(64, 1, strides=2)(residual)    # halve the residual too
x = layers.add([x, residual])
```

So the rule is a pair: `padding="same"` **inside the block,** `strides` **on the projection** to match whatever downsampling the block performs.

SECTION 9.3.2

The same budget, spent two ways



Roughly the same cost. The one on the right can build a representation on top of a representation, four times over.

Parameter counts are computed from the layer sizes shown, not chosen to make the point.

SECTION 9.2

Why "deep and narrow beats broad and shallow"

Broad and shallow

Few layers, each very wide. Many parameters, but only one or two levels of representation — closer to the *shallow learning* of chapter 1.

Deep and narrow

Many layers, each modest. A **long hierarchy of representations**, which is what deep learning is for — and now trainable, thanks to the shortcuts.

03

Batch normalization

Normalising not just the input, but every intermediate activation.

SECTION 9.3

The question that motivates it

the familiar normalisation

PYTHON

```
normalized_data = (data - np.mean(data, axis=...)) / np.std(data, axis=...)
```

> Even if the data entering a `Dense` or `Conv2D` layer has zero mean and unit variance, there is **no reason to expect the same of what comes out**. So: **could normalising intermediate activations help?**

SECTION 9.3

What BatchNormalization does

PHASE	WHAT IT NORMALISES WITH
Training	The mean and variance of **the current batch**.
Inference	An **exponential moving average** of the batchwise mean and variance seen during training — because a big enough representative batch may not be available at inference time.

That moving average is exactly the **non-trainable weight** chapter 7 mentioned: `BatchNormalization` is the only built-in Keras layer that has one.

SECTION 9.3

Nobody is quite sure why it works

> *You'll find that this is true of many things in deep learning — deep learning is not an exact science but a set of ever-changing, empirically derived engineering best practices.*
>
> Chollet & Watson, section 9.3

Worth saying aloud in a classroom: **"we use it because it measurably helps"** is a legitimate engineering answer, and it is **the honest one here**

SECTION 9.3

Where to put it, and one detail that follows

the usual ordering

PYTHON

```python
x = layers.Conv2D(32, 3, use_bias=False)(x)     # no bias here
x = layers.BatchNormalization()(x)
x = layers.Activation("relu")(x)
```

`use_bias=False` because batch normalisation **already centres the output** — the layer's own bias term would be immediately subtracted away, so it is **pure waste**

SECTION 9.3

And why chapter 8 said not to unfreeze it

> During fine-tuning, unfreezing a `BatchNormalization` layer lets its moving statistics be overwritten by a **small, unrepresentative** batch of your new data — undoing the calibration the pretrained model arrived at.

That is the reason behind the rule you were given in chapter 8 without explanation. **Leave batch normalisation frozen when fine-tuning.**

04

Depthwise separable convolutions

A stronger prior, and far fewer parameters.

The assumption it encodes

> That **spatial locations in intermediate activations are highly correlated, but different channels are highly independent**. That holds generally for the representations deep networks learn, so it serves as **a useful prior**

And the general principle behind it: **a model with stronger priors about the structure of its information is a better model — as long as the priors are accurate.**

SECTION 9.4

How the operation splits in two

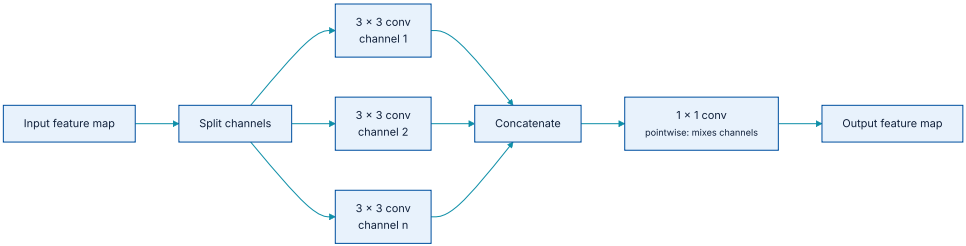


Figure 9.4 — a depthwise convolution (independent spatial convolutions per channel) followed by a pointwise 1×1 convolution that mixes channels.

SECTION 9.4

The parameter arithmetic

OPERATION	PARAMETER COUNT	ARITHMETIC
Regular convolution	**36,864**	$3 \times 3 \times 64 \times 64$
Depthwise separable	**4,672**	$3 \times 3 \times 64 + 64 \times 64$

Roughly **eight times fewer**, with comparable representational power — and the gap **widens as filters or windows get larger**. Smaller models that converge faster and overfit less.

SECTION 9.4

So it must be much faster. It is not.

> Despite repeated requests to NVIDIA, depthwise separable convolutions have not received nearly the same optimisation effort, so they remain **about as fast as regular convolutions** — while using **quadratically fewer parameters and operations**

Use them anyway: the lower parameter count means less overfitting risk, and the channel-independence assumption gives faster convergence and more robust representations.

SECTION 9.4

The wider lesson: hardware, software, and algorithms co-evolve

> Experiment with gradient-free optimisation or spiking neural networks and your first CUDA implementations would be **orders of magnitude slower** than a plain ConvNet, **no matter how good the idea**. Convincing others to adopt it would be a hard sell even if it were plainly better.

GPUs and CUDA enabled backprop-trained ConvNets → NVIDIA optimised for them → the research community consolidated behind them. **Taking a different path now would require a multiyear re-engineering of the whole ecosystem.**

05

Putting it together

A mini Xception, from the four practices.

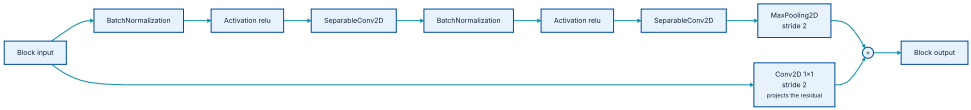
SECTION 9.5

The six principles, collected

1. Organise the model into **repeated blocks** of layers — usually several convolutions plus a max pooling layer.
2. **Filter count should increase as feature-map size decreases.**
3. **Deep and narrow beats broad and shallow.**
4. **Residual connections** around blocks let you train deeper networks.
5. **Batch normalisation** after convolution layers is often beneficial.
6. **`SeparableConv2D` instead of `Conv2D`** is more parameter efficient.

SECTION 9.5

One block, with all four practices in it



Batch norm, separable convolution, pooling, and a projected residual — repeated five times with growing depth.

SECTION 9.5

. . . and the whole model is a loop

the mini Xception

PYTHON

```python
inputs = keras.Input(shape=(180, 180, 3))
x = layers.Rescaling(1.0 / 255)(inputs)

# A REGULAR Conv2D first: the separable assumption ("channels are largely
# independent") is false for RGB - red, green and blue are highly correlated.
x = layers.Conv2D(filters=32, kernel_size=5, use_bias=False)(x)

for size in [32, 64, 128, 256, 512]:
    residual = x
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.SeparableConv2D(size, 3, padding="same", use_bias=False)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.SeparableConv2D(size, 3, padding="same", use_bias=False)(x)
    x = layers.MaxPooling2D(3, strides=2, padding="same")(x)
    residual = layers.Conv2D(size, 1, strides=2, padding="same", use_bias=False)(residual)
    x = layers.add([x, residual])
```

That first-layer detail is easy to miss and worth stating: **separable convolutions are wrong for the raw RGB input**, because **colour channels in natural images are strongly correlated**

The result: fewer parameters, better accuracy

721,857
trainable parameters, mini Xception

1,569,089
trainable parameters, chapter 8's model

90.8%
test accuracy, mini Xception

83.9%
test accuracy, chapter 8's model

Less than half the parameters, seven points better. Following architecture best practices has **an immediate, sizeable effect**.

And note what was not done

> Systematic hyperparameter tuning is **chapter 18**. The point here is that **seven points were available before tuning even started**

SECTION 9.5

Reusing chapter 8's setup unchanged

the unchanged surroundings

PYTHON

```
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.25)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)

model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
history = model.fit(augmented_train_dataset, epochs=100,
                    validation_data=validation_dataset, callbacks=callbacks)
```

A useful habit that falls out of this: **keep the data pipeline and the model definition separable**, so architecture experiments cost **one edit, not a rewrite**

06

Beyond convolution

Vision Transformers, and when they are the wrong choice.

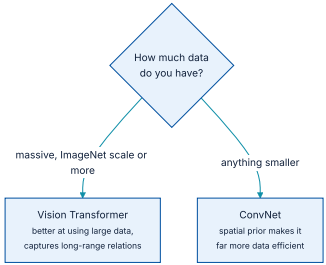
SECTION 9.6

What a Vision Transformer does

- It **splits the image into a 1D sequence of patches**.
- Each patch becomes a flat vector.
- The vector sequence is processed like a sentence — which lets it capture **long-range relationships** between distant parts of the image, **something ConvNets can struggle with**.

SECTION 9.6

When to reach for one



The authors' general experience, stated as a decision.

- ViTs **lack the spatial prior** of ConvNets — the patch-based 2D structure encodes assumptions about local visual structure, which makes ConvNets more data efficient.
- For ViTs to shine **they need to be really large**, which makes them unwieldy for anything smaller than ImageNet.

SECTION 9.6

The state of the contest

> **The battle for image recognition supremacy is far from over**, and ViTs have opened a new chapter. It may be that they replace ConvNets in the long term.

You will most likely meet the architecture in **large-scale generative image models** — chapter 17. For small-scale classification, **ConvNets remain the better bet**

SUMMARY

What has to survive this chapter

1. **Architecture is a choice of hypothesis space** — it encodes prior knowledge, and bad choices cannot be trained away.
2. **Modularity, hierarchy, reuse.** Repeated blocks, and a pyramid of growing depth against shrinking size.
3. **Residual connections** defeat vanishing gradients; project with a 1×1 convolution when shapes differ.
4. **Batch normalisation** after convolutions, with `use_bias=False` — and leave it frozen when fine-tuning.
5. **Depthwise separable convolutions** encode a stronger prior with ~8× fewer parameters, even though the GPU will not run them faster.

SUMMARY

. . . and the number worth remembering

> Assembling those four practices into a mini Xception took the same dogs-vs-cats problem from **83.9% to 90.8%** while **halving the parameter count** — before any hyperparameter tuning.

NOTEBOOK

03_mini_xception.ipynb

NEXT

Chapter 10 — Interpreting what ConvNets learn